



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Цифрова схемотехніка

Лабораторна робота №6

«Пристрої для перетворення чисел. Типові вузли комп'ютера»

Виконав: студент 2 курсу ФІОТ, гр. ІО-41

Давидчук А. М.

Залікова книжка № 4106

Перевірив: *Нікольський С. С.*

Тема: «Пристрої для перетворення чисел. Типові вузли комп'ютера».

Хід роботи

Мій номер залікової книжки: 4106, що у двійковому вигляді є 0001 0000 0000 1010. Звідси $h_7 = 0, h_6 = 0, h_5 = 0, h_4 = 1, h_3 = 0, h_2 = 1, h_1 = 0$. Я студент групи ІО-41, звідси згідно із таблицями, я маю реалізувати 6-розрядних суматор:

Номер варіанта	Розрядність суматора
000	8
001	7
010	6
011	5
100	8
101	5
110	7
111	6

Табл. 1: Визначення розрядності суматора за номером варіанта

Номер групи	Розряди номера ЗК
ІО-41	$h_1 h_2 h_3$
ІО-42	$h_3 h_2 h_1$
ІО-43	$h_2 h_4 h_1$
ІО-44	$h_3 h_1 h_2$
ІО-45	$h_4 h_2 h_3$
ІО-46	$h_4 h_2 h_1$

Табл. 2: Визначення номера варіанта за номером групи

Для побудови 6-розрядного суматора спочатку було реалізовано однорозрядний повний суматор. Він має три входи: a , b , cin , та два виходи: sum і $cout$. Вихід sum відповідає результату додавання поточного розряду, а $cout$ — переносу в наступний розряд.

```
1 module full_adder (  
2     input wire a,  
3     input wire b,  
4     input wire cin,  
5     output wire sum,  
6     output wire cout  
7 );  
8  
9 assign sum = a ^ b ^ cin;  
10 assign cout = (a & b) | (cin & (a ^ b));  
11  
12 endmodule
```

На основі однорозрядного повного суматора було побудовано 6-розрядний суматор структурним способом. Структурний опис означає, що весь пристрій складається з декількох екземплярів нижчого модуля, з'єднаних між собою сигналами переносу. Вихід переносу кожного розряду подається на вхід переносу наступного розряду.

```
1 module my_sum6 (  
2     input wire [5:0] Ain,  
3     input wire [5:0] Bin,  
4     input wire      Ci,  
5     output wire [5:0] Sout,  
6     output wire      Co  
7 );  
8  
9 wire [4:0] c;  
10  
11 full_adder fa0 (  
12     .a  (Ain[0]),  
13     .b  (Bin[0]),  
14     .cin (Ci),  
15     .sum (Sout[0]),  
16     .cout(c[0])  
17 );  
18  
19 full_adder fa1 (  
20     .a  (Ain[1]),  
21     .b  (Bin[1]),  
22     .cin (c[0]),  
23     .sum (Sout[1]),  
24     .cout(c[1])  
25 );  
26  
27 full_adder fa2 (  
28     .a  (Ain[2]),  
29     .b  (Bin[2]),  
30     .cin (c[1]),  
31     .sum (Sout[2]),  
32     .cout(c[2])  
33 );  
34  
35 full_adder fa3 (  
36     .a  (Ain[3]),  
37     .b  (Bin[3]),  
38     .cin (c[2]),  
39     .sum (Sout[3]),  
40     .cout(c[3])  
41 );  
42  
43 full_adder fa4 (  
44     .a  (Ain[4]),  
45     .b  (Bin[4]),  
46     .cin (c[3]),  
47     .sum (Sout[4]),  
48     .cout(c[4])  
49 );
```

```

50
51 full_adder fa5 (
52     .a    (Ain[5]),
53     .b    (Bin[5]),
54     .cin  (c[4]),
55     .sum  (Sout[5]),
56     .cout (Co)
57 );
58
59 endmodule

```

Для перевірки коректності роботи було створено еталонну модель 6-розрядного суматора. На відміну від структурної реалізації, вона використовує поведінковий опис: результат обчислюється вбудованою арифметичною операцією додавання

$$Ain + Bin + Ci$$

Після цього молодші 6 біт записуються у вихідну суму *Sout*, а старший біт використовується як вихід переносу *Co*.

```

1  module ref_sum6 (
2      input  wire [5:0] Ain,
3      input  wire [5:0] Bin,
4      input  wire      Ci,
5      output wire [5:0] Sout,
6      output wire      Co
7  );
8
9  wire [6:0] temp;
10
11 assign temp = Ain + Bin + Ci;
12 assign Sout = temp[5:0];
13 assign Co   = temp[6];
14
15 endmodule

```

Для тестування було створено testbench, який підключає одразу дві моделі:

- my_sum6 — власний 6-розрядний суматор;
- ref_sum6 — еталонний 6-розрядний суматор.

У testbench задаються тестові значення для входів:

- Ain — перший 6-бітний доданок;
- Bin — другий 6-бітний доданок;
- Ci — вхідний перенос.

Результати роботи позначаються так:

- res_my — сума, отримана структурним суматором;
- cm — перенос, отриманий структурним суматором;

- `res_ref` — сума, отримана еталонною моделлю;
- `cr` — перенос, отриманий еталонною моделлю.

Якщо для всіх тестових наборів виконується:

$$res_my = res_ref, \quad cm = cr,$$

то це означає, що структурний суматор працює правильно.

```

1  module test_sum6;
2
3  reg  [5:0] Ain_r, Bin_r;
4  reg          Ci_r;
5
6  wire [5:0] Ain, Bin;
7  wire          Ci;
8
9  wire [5:0] res_my, res_ref;
10 wire          cm, cr;
11
12 assign Ain = Ain_r;
13 assign Bin = Bin_r;
14 assign Ci  = Ci_r;
15
16 my_sum6 my_block (
17     .Ain (Ain),
18     .Bin (Bin),
19     .Ci  (Ci),
20     .Sout(res_my),
21     .Co  (cm)
22 );
23
24 ref_sum6 ref_block (
25     .Ain (Ain),
26     .Bin (Bin),
27     .Ci  (Ci),
28     .Sout(res_ref),
29     .Co  (cr)
30 );
31
32 initial
33 begin
34     $display("Time\tAin\tBin\tCi\tres_my\tcm\tres_ref\tcr");
35     $monitor("%0t\t%b\t%b\t%b\t%b\t%b\t%b\t%b",
36             $time, Ain, Bin, Ci, res_my, cm, res_ref, cr);
37 end
38
39 initial
40 begin
41     Ain_r = 6'b000000;
42     Bin_r = 6'b000000;
43     Ci_r  = 1'b0;
44
45     #10 Ain_r = 6'b000001; Bin_r = 6'b000010; Ci_r = 1'b0; // 1 + 2

```

```

46     #10 Ain_r = 6'b000101; Bin_r = 6'b000011; Ci_r = 1'b0; // 5 + 3
47     #10 Ain_r = 6'b001111; Bin_r = 6'b000001; Ci_r = 1'b0; // 15 + 1
48     #10 Ain_r = 6'b010101; Bin_r = 6'b001010; Ci_r = 1'b1; // 21 + 10 + 1
49     #10 Ain_r = 6'b111111; Bin_r = 6'b000001; Ci_r = 1'b0; // overflow
50     #10 Ain_r = 6'b101010; Bin_r = 6'b010101; Ci_r = 1'b1;
51     #10 Ain_r = 6'b111100; Bin_r = 6'b000011; Ci_r = 1'b1;
52     #10 Ain_r = 6'b100000; Bin_r = 6'b100000; Ci_r = 1'b0;
53     #10 Ain_r = 6'b011111; Bin_r = 6'b011111; Ci_r = 1'b1;
54     #10 $finish;
55 end
56
57 endmodule

```

Для автоматичного запуску компіляції та моделювання в середовищі ModelSim було використано файл Stim.do.

```

1  vlib work
2
3  vlog full_adder.v
4  vlog sum6bits.v
5  vlog ref_sum6bits.v
6  vlog test.v
7
8  vsim work.test_sum6
9
10 add wave sim:/test_sum6/Ain
11 add wave sim:/test_sum6/Bin
12 add wave sim:/test_sum6/Ci
13 add wave sim:/test_sum6/res_my
14 add wave sim:/test_sum6/cm
15 add wave sim:/test_sum6/res_ref
16 add wave sim:/test_sum6/cr
17
18 run -all

```

У testbench було подано декілька характерних наборів вхідних даних, зокрема:

- додавання невеликих чисел без переносу;
- додавання з вхідним переносом;
- випадки переповнення 6-розрядної сітки;
- граничні випадки, коли використовуються максимальні значення.

Наприклад:

$$010101_2 + 001010_2 + 1 = 100000_2$$

тобто

$$21 + 10 + 1 = 32.$$

Інший приклад:

$$111111_2 + 000001_2 + 0 = 1000000_2$$

Оскільки суматор є 6-розрядним, у вихідну шину суми записуються лише молодші 6 біт:

$$S_{out} = 000000,$$

а старший біт переходить у сигнал переносу:

$$Co = 1.$$

Результати моделювання наведено на часовій діаграмі.

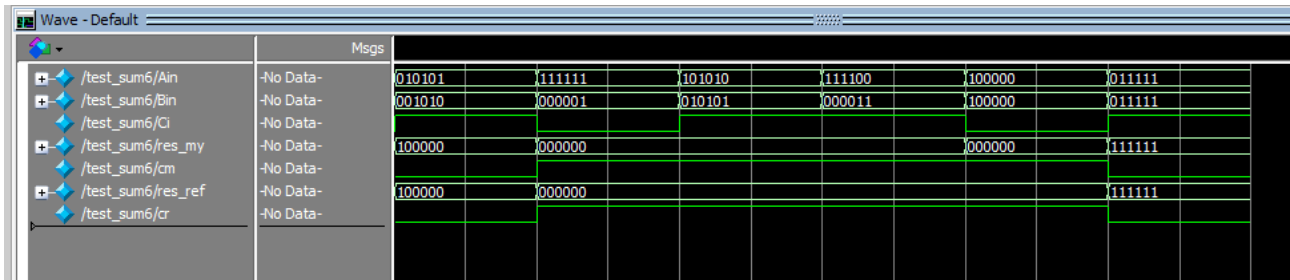


Рис. 1: Часова діаграма роботи 6-розрядного структурного та еталонного суматорів

На часовій діаграмі видно, що при зміні значень входів Ain, Bin та Ci результати res_my і res_ref повністю збігаються. Так само збігаються сигнали переносу cm і cr. Це підтверджує, що структурна реалізація 6-розрядного суматора функціонує коректно та дає ті самі результати, що і поведінкова еталонна модель.

Висновок

У ході виконання лабораторної роботи було реалізовано 6-розрядний суматор мовою Verilog. Для побудови пристрою використано структурний підхід: суматор було зібрано з однорозрядних повних суматорів, з'єднаних каскадно через сигнали переносу. Також було створено еталонну модель суматора на поведінковому рівні та testbench для подачі тестових вхідних даних.

У процесі моделювання в середовищі ModelSim було перевірено роботу суматора на декількох наборах вхідних даних, включаючи звичайні випадки додавання, додавання з вхідним переносом і випадки переповнення. За результатами симуляції встановлено, що сигнали суми та переносу структурної реалізації повністю збігаються з відповідними сигналами еталонної моделі. Отже, реалізований 6-розрядний суматор працює правильно.